

<https://cassandra.apache.org/doc/stable/cassandra/cql/>
https://docs.datastax.com/en/cql-oss/3.1/cql/cql_intro_c.html

#-# Data types

http://docs.datastax.com/en/cql/3.3/cql/cql_reference/cql_data_types_c.html

Type	Example
-----	-----
numeric	bigint, decimal, double, float, int, varint, counter
string	ascii, text, varchar
date	timestamp, timeuuid
other	boolean, uuid, inet, blob

collections:

- SET<int> A set allows only unique values and does not preserve order.
- LIST<int> A list allows duplicate values and preserves order.
- MAP<text,timestamp> A map is a dictionary of key value pairs.
- TUPLE<int,text,varchar,double,int> A tuple can have many different data types, but they must always appear in the same order.
- Complex types can be nested. e.g. map<text,frozen<tuple<int,double>>>. Whenever a complex type is nested, it must be frozen. Nested complex types are written and read as blobs, so they are not particularly efficient.

User Defined Types

- CREATE TYPE clip (name varchar, duration int);
- CREATE TYPE person (name varchar, id varchar);
- CREATE TABLE courses (id text, author frozen<person>);

User defined types must always use the frozen keyword.

User defined types have the following advantages over using a tuple:

- Ability to identify components by name, not just order.
- Can be very helpful when multiple components have the same type.
- Can be used across multiple tables.

#-# Naming Conventions / Rules

Best advice: Don't get creative. Stick with alpha, lowercase, snake-case names.

- Snake-case is the preferred naming style in Cassandra (e.g. my_table)
- No hyphens (e.g. my-table)
- No spaces (e.g. my table)
- Double quotes are required if something starts with a number (e.g. "2050predictions")
- Double quotes are required if mixed case names are used (e.g. "myTable")

#-# Data definition queries

CREATE: Used to create a keyspace or table.

CREATE KEYSPACE sample WITH REPLICATION =
{'class':'SimpleStrategy','replication_factor':3};

CREATE TABLE my_table (id text, size int, PRIMARY KEY (id));

INSERT: Used to update or insert table data.

INSERT INTO my_table (id) VALUES ('guid');

DROP: Drop / delete a table or keyspace.

DROP KEYSPACE sample;

DROP TABLE my_table;

Examples

```
CREATE KEYSPACE pluralsight WITH REPLICATION =  
{'class':'SimpleStrategy','replication_factor':1};
```

```
CREATE TABLE courses (id varchar PRIMARY KEY);  
CREATE TABLE IF NOT EXISTS courses (id VARCHAR PRIMARY KEY);  
DROP TABLE courses;
```

```
CREATE TABLE courses (id varchar PRIMARY KEY, name varchar, author varchar, audience int,  
duration int, hasClosedCaptions boolean, released timestamp);
```

```
SELECT id, title FROM pluralsight.courses;  
SELECT title, duration AS length FROM pluralsight.courses WHERE id = 'cassandra-  
developers';  
SELECT title, published FROM pluralsight.courses WHERE id IN ('cassandra-developers',  
'node-intro');  
SELECT * FROM pluralsight.courses LIMIT 100;
```

```
INSERT INTO pluralsight.courses (id, author) VALUES ('cassandra-developers', 'paul-ofallon');
```

```
SELECT id, WRITETIME(author) FROM pluralsight.courses;
```

-- Working with Collections

-- SET<type>

```
INSERT INTO courses (id, name, features) VALUES ('node-intro', 'Introduction to Node.js',  
{'cc', 'transcript'});
```

-- LIST<type>

```
INSERT INTO courses (id, clips) VALUES ('node-intro', ['Getting Started']);
```

-- MAP<key-type,value-type>

```
INSERT INTO users (id, last_login) VALUES ('john-doe', {'abcde': '2015-06-30 09:02:24'});
```

-- Secondary indexes

```
CREATE TABLE users (id varchar, first_name varchar, tags set<varchar>, PRIMARY KEY (id));  
CREATE INDEX ON users(tags);  
INSERT INTO users (id, first_name, tags) VALUES ('john-doe','John', {'java'});  
SELECT * FROM users WHERE tags CONTAINS 'java';
```

-- A secondary index for a map

```
CREATE INDEX ON table(KEYS(map_column));  
SELECT * FROM table WHERE map_column CONTAINS KEY 'foo';
```